



Politechnika Łódzka

Instytut Automatyki

Zakład sterowania robotów

TWORZENIE SYSTEMÓW ROBOTYCZNYCH

Temat 4

ROS2 Action

20 marca 2025

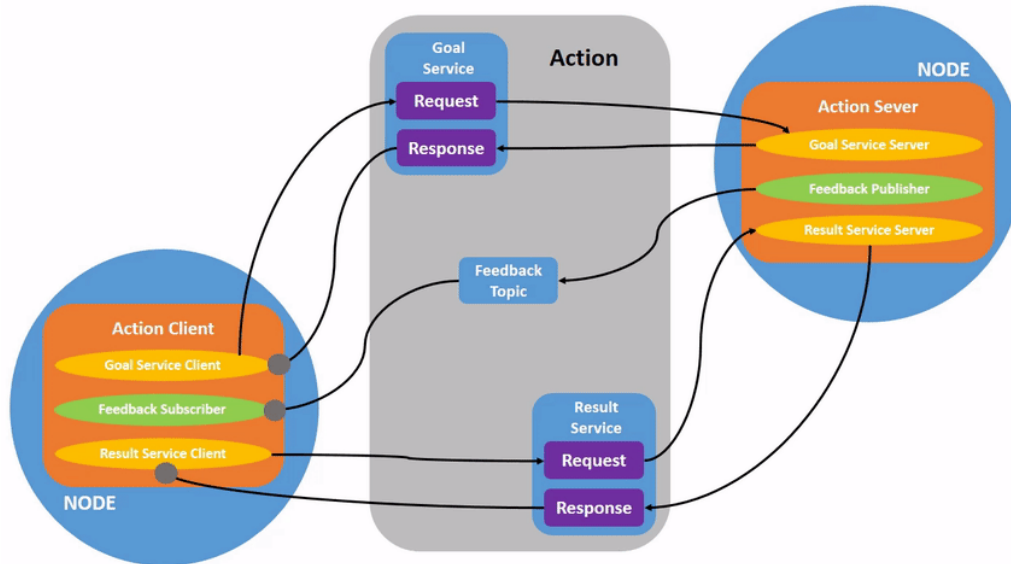


Spis treści

1	Wprowadzenie	2
1.1	Przykład 1 - Pobieranie dużego pliku z internetu	2
1.2	Przykład 2 - Robot mobilny jadący do wyznaczonego punktu	3
2	Struktura wiadomości dla akcji	3
2.1	Jak sprawdzić typ wiadomości dla akcji w ROS2?	4
2.2	Procedura dodania nowej wiadomości dla akcji (.action) do pakietu	4
2.2.1	Przejsć do pakietu <code>example_tsr_msgs</code>	4
2.2.2	Utworzenie podkatalogu <code>action</code>	5
2.2.3	Utworzenie pliku z wiadomością <code>.action</code>	5
2.2.4	Dodanie interfejsu do <code>CMakeLists.txt</code>	5
2.2.5	Modyfikacja <code>package.xml</code>	6
2.2.6	Budowanie pakietu	7
2.2.7	Weryfikacja działania	7
3	Procedura tworzenia nowego serwera dla akcji	8
3.1	Przejsć do lokalizacji pakietu <code>example_tsr_project</code>	8
3.2	Utworzenie pliku <code>go_to_pose_action_server.py</code>	8
3.3	Określić typ wiadomości dla serwera akcji	8
3.4	Napisanie kodu dla węzła	9
3.5	Aktualizacja <i><code>setup.py</code></i>	10
3.6	Budowanie paczki	11
3.7	Weryfikacja działania	11
4	Procedura tworzenia nowego klienta dla akcji	12
4.1	Przejsć do lokalizacji pakietu <code>example_tsr_project</code>	12
4.2	Utworzenie pliku <code>go_to_pose_action_client.py</code>	12
4.3	Sprawdzić typ wiadomości obsługiwanej przez serwer akcji	13
4.4	Napisanie kodu dla węzła	13
4.5	Aktualizacja <i><code>setup.py</code></i>	14
4.6	Budowanie paczki	15
4.7	Weryfikacja działania	15
5	Dokumentacja i inne linki	16

1 Wprowadzenie

ROS2 Action (Akcje) są kolejnym sposobem wymiany informacji między węzłami. Akcje są stosowane, gdy wykonywane zadanie zajmuje więcej czasu i wymaga monitorowania postępu lub możliwości anulowania wysłanej akcji.



Rys. 1.1: Komunikacja dla akcji. Animacja dostępna na <https://docs.ros.org/en/foxy/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Actions/Understanding-ROS2-Actions.html>

Akcja składa się z trzech głównych elementów:

- Goal (cel) – Klient wysyła żądanie wykonania zadania do serwera akcji.
- Feedback (informacja zwrotna) – Serwer akcji może okresowo wysyłać klientowi informacje o postępie.
- Result (wynik) – Po zakończeniu zadania serwer zwraca końcowy rezultat.

Komunikacja opiera się na wzorcu **Klient-Serwer**:

- Klient wysyła żądanie (goal) do Serwera Akcji.
- Serwer rozpoczyna realizację zadania i może wysyłać feedback.
- Po zakończeniu zadania serwer wysyła końcowy wynik.

Dodatkowo, klient może w każdej chwili anulować zadanie, jeśli np. nie jest już potrzebne.

1.1 Przykład 1 - Pobieranie dużego pliku z internetu

Pobieranie dużych plików może zajmować wiele godzin. Z tego powodu użytkownik końcowy chce monitorować postęp pobierania pliku.

- Goal (Cel) - Kliknięcie przycisku „Pobierz” na stronie internetowej wywołuje akcję pobierania pliku.
- Feedback (Informacja zwrotna) - Przeglądarka pokazuje pasek postępu: „10% pobrane”, „50% pobrane”, „80% pobrane”.
- Result (Wynik) - Plik zostaje pobrany i zapisany na dysku.

Użytkownik anulować pobieranie pliku, jeśli zobaczy, że plik jest za duży lub pobiera niewłaściwy plik.

1.2 Przykład 2 - Robot mobilny jadący do wyznaczonego punktu

- Goal (Cel) - Robot otrzymuje polecenie, aby pojechać do określonego punktu (np. współrzędne X, Y w przestrzeni).
- Feedback (Informacja zwrotna) - Robot raportuje działania:
 - Planowanie trasy do punktu (X, Y)
 - Rozpoczęcie jazdy
 - Przejechano 10% trasy
 - Przejechano 50% trasy
 - Wykryto przeszkodę. Nastąpiło wygenerowanie nowej trasy
- Result (Wynik) - Robot osiąga docelową pozycję i informuje o zakończeniu zadania.

Można przerwać zadanie w dowolnym momencie, np. zmieni się cel misji.

2 Struktura wiadomości dla akcji

Dla komunikacji ROS2 Action żądanie wykonania akcji wysyłane jest przez klienta na serwer akcji. Każda wiadomość posiada predefiniowane pola i typy danych, zdefiniowane w plikach `.action`. Wykorzystywane typy pól w wiadomości są takie same jak dla ROS Topic. Różnica pomiędzy wiadomościami dla ROS Topic a ROS Action związana jest z konstrukcją wiadomości. Jest ona podobna bardziej do ROS Service, gdyż składa się z — — —. Nad pierwszym znakiem — — — znajduje się żądanie wysyłane przez klienta. Poniżej znajduje się sekcja odpowiedzialna za wysyłanie ostatecznego rezultatu a ostatnia sekcja wiadomości dotyczy informującej o postępie.

Struktura wiadomości dla akcji (`.action`)

Plik `.action` składa się z trzech sekcji:

- Goal (cel akcji) – żądanie wykonania akcji wysyłane do serwera obsługującego akcję
- — — —
- Result (wynik końcowy) – odpowiedź serwera akcji dotycząca ostatecznego wyniku działania akcji
- — — —
- Feedback (informacja zwrotna o postępie) - Wysyłane co jakiś czas informacje o postępie wykonania akcji.

Przykład wiadomości dla akcji

```
1 # Goal (cel akcji)
2 int32 target_x
3 int32 target_y
4 ---
5 # Result (wynik końcowy)
6 bool success
7 ---
8 # Feedback (informacja zwrotna o postępie)
9 float32 distance_remaining
```

Opis pól:

- Goal - Definiuje dane wejściowe, czyli cel działania (np. współrzędne docelowe dla robota).
- Result - Opisuje wynik końcowy, np. czy robot dotarł do celu.

- Feedback - Zawiera informacje o postępie (np. ile jeszcze drogi pozostało).

Przykładowe wywołanie akcji w ROS2

Użytkownik wysyła żądanie dojazdu do punktu (Goal):

```
1 int32 target_x
2 int32 target_y
```

Serwer akcji zwraca informację o postępie (Feedback):

```
1 float32 distance_remaining
```

Po wykonaniu akcji serwer akcji zwraca wynik działania akcji (Result):

```
1 bool success
```

2.1 Jak sprawdzić typ wiadomości dla akcji w ROS2?

W terminalu można zobaczyć listę aktywnych akcji używanych w systemie:

```
1 ros2 action list
```

Sprawdzenie typu wiadomości dla akcji o podanej nazwie `/example_action`. Polecenie należy wywołać z flagą `-t` aby wyświetlił się typ.

```
1 ros2 action info -t /example_action
```

W informacji o akcji wyświetlony jest jej typ. Informacja o typie może być następująca `turtlesim/action/RotateAbsolute`

```
1 ros2 interface show turtlesim/action/RotateAbsolute
```

Przykładowy wynik:

```
1 # The desired heading in radians
2 float32 theta
3 ---
4 # The angular displacement in radians to the starting position
5 float32 delta
6 ---
7 # The remaining rotation in radians
8 float32 remaining
```

2.2 Procedura dodania nowej wiadomości dla akcji (.action) do pakietu

Jest to tylko przypomnienie z wprowadzenia. Podane pliki i katalogi istnieją już w tym pakiecie `example_tsr_msgs`. Procedura natomiast zostanie tak opisana jakby ich nie było. Zakładamy, że dodajemy wiadomości do już istniejącego pakietu, który może nie być skonfigurowany do generowania nowych wiadomości. Może brakować folderów.

2.2.1 Przejść do pakietu `example_tsr_msgs`

Należy w terminalu przejść do workspace'a (`example_tsr_ws`) a następnie do `src` i pakietu `example_tsr_msgs`:

```
1 cd ~/tsr_workspaces/example_tsr_ws/src/example_tsr_msgs
```

2.2.2 Utworzenie podkatalogu action

Utworzenie w pakiecie `example_tsr_msgs` katalogów dla interfejsów komunikacyjnych dla ROS Action (katalog `action`)

W tym celu należy przejść do lokalizacji głównego katalogu dla pakietu a następnie utworzyć katalog `action`. Można dodać ręcznie katalog z pominięciem terminala.

```
1 cd ~/tsr_workspaces/example_tsr_ws/src/example_tsr_msgs
2 mkdir action
```

2.2.3 Utworzenie pliku z wiadomością .action

W katalogu `action` (`~/tsr_workspaces/example_tsr_ws/src/example_tsr_msgs/action`) utworzyć plik tekstowy `GoToPose.action`, w którym zdefiniowany zostanie interfejs komunikacyjny dla wiadomości ROS Action. Wszystkie pliki w tym katalogu będą miały rozszerzenie `.action`. Do pliku należy wkleić przykład struktury wiadomości:

```
1 turtlesim/Pose goal_pose # Współrzędne docelowe
2 ---
3 bool success # Czy pomyślnie zakończono dojazd do celu
4 string message # Komunikat o wyniku
5 ---
6 float32 distance_remaining # Odległość do celu
```

2.2.4 Dodanie interfejsu do CMakeLists.txt.

W głównym katalogu pakietu `example_tsr_ws_msgs` znajduje się plik `CMakeLists.txt`, który należy otworzyć w celu edycji.

Do pliku należy dodać następujące linie, jeśli brakują:

```
1 find_package(action_msgs REQUIRED)
2 find_package(rosidl_default_generators REQUIRED)
3
4 rosidl_generate_interfaces(${PROJECT_NAME}
5 "action/GoToPose.action"
6 DEPENDENCIES action_msgs
7 )
8
9 ament_export_dependencies(rosidl_default_runtime)
```

W linii 1 należy wskazać zależność od `action_msgs`, gdyż wykorzystujemy pakiet odpowiedzialny za wygenerowanie wiadomości dla akcji. Ponadto niezbędne jest zaimportowanie pakietu `rosidl_default_generators` (linia 2).

W części `rosidl_generate_interfaces` (linie 4-6) należy dodać utworzone interfejsy komunikacyjne oraz zależności z nimi związane. Należy również wyeksportować zależności od `message runtime` (linia 8)

Po edycji plik powinien wyglądać następująco (Rys. 2.1):

```

1 cmake_minimum_required(VERSION 3.8)
2 project(example_tsr_msgs)
3
4 if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID MATCHES "Clang")
5   add_compile_options(-Wall -Wextra -Wpedantic)
6 endif()
7
8 # find dependencies
9 find_package(action_msgs REQUIRED)
10 find_package(ament_cmake REQUIRED)
11 find_package(geometry_msgs REQUIRED)
12 find_package(turtlesim REQUIRED)
13 find_package(rosidl_default_generators REQUIRED)
14
15 rosidl_generate_interfaces(${PROJECT_NAME}
16   "msg/ExampleMsgType.msg"
17   "srv/ExampleSrvType.srv"
18   "srv/AddGoToPose.srv"
19   "action/GoToPose.action"
20   DEPENDENCIES action_msgs geometry_msgs turtlesim
21 )
22
23 if(BUILD_TESTING)
24   find_package(ament_lint_auto REQUIRED)
25   # the following line skips the linter which checks for copyrights
26   # comment the line when a copyright and license is added to all source files
27   set(ament_cmake_copyright_FOUND TRUE)
28   # the following line skips cpplint (only works in a git repo)
29   # comment the line when this package is in a git repo and when
30   # a copyright and license is added to all source files
31   set(ament_cmake_cpplint_FOUND TRUE)
32   ament_lint_auto_find_test_dependencies()
33 endif()
34
35 ament_export_dependencies(rosidl_default_runtime)
36 ament_package()

```

Rys. 2.1: Plik CMakeLists.txt po edycji dla paczki example_tsr_msgs

2.2.5 Modyfikacja package.xml

Następnie należy dokonać edycji pliku package.xml i dodać następujące zależności, jeśli nie są dodane.

```

1 <depend>action_msgs</depend>
2 <buildtool_depend>rosidl_default_generators</buildtool_depend>
3 <exec_depend>rosidl_default_runtime</exec_depend>
4 <member_of_group>rosidl_interface_packages</member_of_group>

```

Linia 1 związana jest z zależnością od pakietu do generowania wiadomości dla akcji. Linia 2 określa narzędzie budowania potrzebne do generowania kodu dla interfejsów komunikacyjnych (msg, srv, action). Linia 3 jest niezbędna do późniejszego wykorzystania interfejsów. rosidl_interface_packages jest nazwą grupy z zależnościami w utworzonym pakiecie.

Po edycji plik powinien wyglądać następująco (Rys. 2.2):

```

1 <?xml version="1.0"?>
2 <?xml-model href="http://download.ros.org/schema/package_format3.xsd" schematypens="http://www.w3.org/2001/XMLSchema"?>
3 <package format="3">
4   <name>example_tsr_msgs</name>
5   <version>0.0.0</version>
6   <description>TODO: Package description</description>
7   <maintainer email="ubuntu@todo.todo">ubuntu</maintainer>
8   <license>TODO: License declaration</license>
9
10  <buildtool_depend>ament_cmake</buildtool_depend>
11
12  <test_depend>ament_lint_auto</test_depend>
13  <test_depend>ament_lint_common</test_depend>
14
15  <depend>geometry_msgs</depend>
16  <depend>turtlesim</depend>
17  <depend>action_msgs</depend>
18
19  <buildtool_depend>rosidl_default_generators</buildtool_depend>
20  <exec_depend>rosidl_default_runtime</exec_depend>
21  <member_of_group>rosidl_interface_packages</member_of_group>
22
23  <export>
24    <build_type>ament_cmake</build_type>
25  </export>
26 </package>

```

Rys. 2.2: Plik package.xml po edycji dla pakietu example_tsr_msgs

2.2.6 Budowanie pakietu

Należy przejść do głównego katalogu dla workspace'a (example_tsr_ws):

```
1 cd ~/tsr_workspaces/example_tsr_ws
```

następnie zbudować pojedynczy pakiet (można też zbudować cały workspace `colcon build`):

```
1 colcon build --packages-select example_tsr_msgs
```

i wykonać ponowny `source` w terminalu (lub uruchomić nowy terminal):

```
1 source install/setup.bash
```

2.2.7 Weryfikacja działania

Dla utworzonego interfejsu dla ROS Services należy sprawdzić czy nowo utworzony interfejs jest widoczny.

```
1 ros2 interface show example_tsr_msgs/action/GoToPose
```

Prawidłowy rezultat przedstawia (Rys. 2.3):

```

ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$ ros2 interface show example_tsr_msgs/action/GoToPose
turtlesim/Pose goal_pose # Współrzędne docelowe
  float32 x
  float32 y
  float32 theta
  float32 linear_velocity
  float32 angular_velocity
---
bool success # Czy pomyślnie zakończono dojazd do celu
string message # Komunikat o wyniku
---
float32 distance_remaining # Odległość do celu
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$

```

Rys. 2.3: Wynik operacji sprawdzenia w terminalu struktury utworzonych interfejsów dla pakietu example_tsr_msgs.

Po użyciu polecenia sprawdzającego strukturę interfejsu nie mogą pojawić się błędy. Struktura ma być zgodna z tą jaką jest w dodanym pliku `GoToPose.action`. Jeśli wyświetla się coś innego tzn., że należy ponownie zweryfikować poprawność konfiguracji i przejść procedurę jeszcze raz zwracając dokładniejszą uwagę na wykonywane kroki.

3 Procedura tworzenia nowego serwera dla akcji

Załóżmy, że mamy już istniejący pakiet (`example_tsr_project`) ROS2 kompilowany jako paczka Python i chcemy do niego dodać serwer akcji odpowiedzialny za dojazd do celu. Jest to zmodyfikowana wersja węzła z ROS Services odpowiedzialnego za dojazd robota do punktów, które są dodawane przez serwis. Tym razem wykorzystana zostanie wiadomość dla akcji typu `GoToPose` z pakietu `example_tsr_msgs`.

3.1 Przejście do lokalizacji pakietu `example_tsr_project`

Przejdź do pakietu, w którym ma zostać utworzony nowy Serwer serwisowy. W naszym przypadku będzie to pakiet z plikami projektowymi `example_tsr_project`.

```
1 cd ~/tsr_workspaces/example_tsr_ws/src/example_tsr_project
```

— `~` - oznacza lokalizację katalogu domowego

— `example_tsr_ws` - utworzony workspace. W katalogu `src` znajdują się wszystkie pakiety (nowe lub pobrane np. z githuba).

3.2 Utworzenie pliku `go_to_pose_action_server.py`

W pakiecie `example_tsr_project` należy utworzyć nowy plik *`go_to_pose_action_server.py`* w katalogu `example_tsr_project` (w kolejnym katalogu o tej samej nazwie a nie w głównym pakiecie gdzie znajdują się pliki `package.xml` i `setup.py`) a następnie nadać uprawnienia do wykonania. Brak uprawnień jest jednym z występujących problemów z widocznością węzła co uniemożliwia jego uruchomienie pomimo poprawnej konfiguracji paczki.

Należy wpisać kolejne polecenia w terminalu (należy znajdować się w głównej lokalizacji pakietu `example_tsr_project`)

```
1 touch example_tsr_project/go_to_pose_action_server.py
2 chmod +x example_tsr_project/go_to_pose_action_server.py
```

Nowo utworzony węzeł z Serwerem serwisowym powinien mieć następującą lokalizację:

```
1 ~/tsr_workspaces/example_tsr_ws/src/example_tsr_project/example_tsr_project/go_to_pose_action_server.py
```

3.3 Określić typ wiadomości dla serwera akcji

Następnie należy utworzyć nowy węzeł z serwerem do obsługi akcji, z którym komunikujemy się poprzez podanie nazwy (`/goto_pose`) serwera akcji. W tym przykładzie do komunikacji zastosowano wiadomość typu `GoToPose` z utworzonego pakietu `example_tsr_msgs`. Do utworzenia Serwera akcji niezbędna jest znajomość interfejsu komunikacyjnego (struktury przesyłanych wiadomości), który można sprawdzić w terminalu w następujący sposób:

```
1 ros2 interface show example_tsr_msgs/action/GoToPose
```

Podobnie jak dla ROS Topic możemy w tym miejscu zastosować dowolny interfejs, ale później należy konsekwentnie wykonać zamianę nazw pakietu, z którego pochodzi wiadomość, zamianę typu i struktury obsługiwanej wiadomości na właściwą. W przypadku braku odpowiedniego interfejsu należy go utworzyć.

W wyniku operacji wyświetla się struktura wiadomości dla ROS Action (Rys. 3.1):

```
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$ ros2 interface show example_tsr_msgs/action/GoToPose
turtlesim/Pose goal_pose # Współrzędne docelowe
  float32 x
  float32 y
  float32 theta
  float32 linear_velocity
  float32 angular_velocity
---
bool success # Czy pomyślnie zakończono dojazd do celu
string message # Komunikat o wyniku
---
float32 distance_remaining # Odległość do celu
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$
```

Rys. 3.1: Struktura wiadomości dla akcji typu GoToPose

Znajomość typów i pól w wiadomości jest niezbędna do jej prawidłowego uzupełnienia.

3.4 Napisanie kodu dla węzła

Następnie należy napisać prosty węzeł z Serwerem akcji (zalecane skopiowanie kodu z dołączonych plików lub ich wstawienie, gdyż zachowują wcięcia przy kopiowaniu):

```
1 import rclpy
2 from rclpy.node import Node
3 from rclpy.action import ActionServer # Import obiektu do obsługi akcji
4 from example_tsr_msgs.action import GoToPose # Import wiadomości dla akcji typu GoToPose z pakietu
  ↳ example_tsr_msgs
5 from turtlesim.msg import Pose
6 from geometry_msgs.msg import Twist
7 from rclpy.executors import MultiThreadedExecutor # Dodanie importu i wielowątkowość była konieczna do
  ↳ prawidłowego odczytu aktualnego położenia robota przez Subscriber pose_subscription podczas realizacji
  ↳ akcji dojazdu do punktu. Rozwiązało to problem z brakiem możliwości wystąpienia kolejnego celu.
8 import math
9 import time
10
11
12 class GoToPoseActionServer(Node):
13     def __init__(self):
14         super().__init__('goto_pose_action_server')
15         # Utworzenie serwera dla akcji o nazwie 'goto_pose', który jest typu GoToPose. execute_callback jest
16         ↳ to argument dla którego podaje się metodę klasy lub inną funkcję, która ma zostać wykonana po
17         ↳ odebraniu zapytania dotyczącego wykonania akcji.
18         self.action_server = ActionServer(
19             self,
20             GoToPose,
21             'goto_pose',
22             execute_callback=self.execute_callback)
23
24         # Publikowanie prędkości
25         self.cmd_vel_publisher = self.create_publisher(Twist, '/turtle1/cmd_vel', 10)
26         # Odbiór pozycji
27         self.pose_subscription = self.create_subscription(Pose, '/turtle1/pose', self.pose_callback, 10)
28         # Aktualna pozycja
29         self.current_pose = Pose()
30         self.get_logger().info("Serwer akcji goto_pose uruchomiony.")
31
32     def pose_callback(self, msg):
33         """Aktualizacja bieżącej pozycji robota"""
34         self.current_pose = msg
35
36     def execute_callback(self, goal_handle):
37         """Obsługuje wykonanie akcji - nawigacja do pozycji"""
38         self.get_logger().info(f"Rozpoczęto dojazd do: ({goal_handle.request.goal_pose.x},
39         ↳ {goal_handle.request.goal_pose.y})")
40
41         goal = goal_handle.request.goal_pose
42         success = False
43
44         # Utworzenie wiadomości dla prędkości i Feedback
45         vel_msg = Twist()
46         feedback_msg = GoToPose.Feedback()
```

```

44
45     # Rozpoczęcie autonomicznego dojazdu do celu
46     while rclpy.ok():
47         distance = self.euclidean_distance(goal)
48
49         if distance >= 0.1:
50             if abs(self.angular_vel(goal)) > 0.2:
51                 vel_msg.angular.z = 0.6
52             else:
53                 vel_msg.linear.x = self.linear_vel(goal)
54                 vel_msg.angular.z = self.angular_vel(goal)
55
56                 self.cmd_vel_publisher.publish(vel_msg)
57         else:
58             success = True
59             vel_msg.linear.x = 0.0
60             vel_msg.angular.z = 0.0
61             self.cmd_vel_publisher.publish(vel_msg)
62             break
63
64         # Wysyłanie feedbacku
65         feedback_msg.distance_remaining = distance
66         goal_handle.publish_feedback(feedback_msg)
67
68         self.get_logger().info(f"Odległość do celu: {distance:.2f}m")
69
70         time.sleep(0.1) # Uniknięcie 100% obciążenia procesora
71
72         # Wysłanie wyniku akcji
73         result = GoToPose.Result()
74         result.success = success
75         result.message = "Robot dotarł do celu!" if success else "Nie udało się dotrzeć do celu."
76         goal_handle.succeed() # Ustawienie zakończenia obsługi akcji
77         self.get_logger().info(result.message)
78
79         return result # odesłanie wyniku
80
81     def euclidean_distance(self, goal_pose):
82         """Odległość euklidesowa do celu"""
83         return math.sqrt(math.pow((goal_pose.x - self.current_pose.x), 2) +
84                           math.pow((goal_pose.y - self.current_pose.y), 2))
85
86     def linear_vel(self, goal_pose, constant=1.5):
87         """Obliczanie prędkości liniowej"""
88         vel = constant * self.euclidean_distance(goal_pose)
89         return min(max(-1.5, vel), 1.5)
90
91     def steering_angle(self, goal_pose):
92         """Obliczanie kąta do sterowania"""
93         return math.atan2(goal_pose.y - self.current_pose.y, goal_pose.x - self.current_pose.x)
94
95     def angular_vel(self, goal_pose, constant=6.0):
96         """Obliczanie prędkości kątowej"""
97         return constant * (self.steering_angle(goal_pose) - self.current_pose.theta)
98
99
100 def main():
101     # Inicjalizacja ROS2 dla programu, musi być wykonana przed utworzeniem jakiegokolwiek węzła (node'a).
102     rclpy.init()
103     # Utworzenie nowego obiektu węzła ROS2 na podstawie klasy GoToPoseActionServer
104     node = GoToPoseActionServer()
105
106     # Użycie MultiThreadedExecutor zamiast spin()
107     executor = MultiThreadedExecutor()
108     executor.add_node(node)
109     executor.spin()
110
111     # Usuwa węzeł przed zamknięciem programu. Zwalnianie zasoby pamięci zajmowane przez ten obiekt.
112     node.destroy_node()
113     # Zamykanie połączenia z ROS2.
114     rclpy.shutdown()
115
116
117 if __name__ == '__main__':
118     main()
119

```

3.5 Aktualizacja *setup.py*

W pakiecie *example_tsr_project* w pliku *setup.py*. Należy zmodyfikować linię z *entry_points* tak, aby dodać nowy węzeł odpowiedzialny za uruchomienie serwera akcji, który został przed chwilą utworzony. W tym miejscu należy dodawać uruchomienie każdego nowego węzła (node).

```

1 entry_points={
2     'console_scripts': [
3         'goto_pose_action_server = example_tsr_project.goto_pose_action_server:main',
4     ],
5 }

```

Lewa strona w linii z dodanym `goto_pose_action_server` odnosi się do unikalnej nazwy w pakiecie, która będzie wykorzystywana podczas uruchamiania nowego węzła. Ta nazwa może nie być widoczna po uruchomieniu węzła na liście uruchomionych węzłów (`ros2 node list`), jeśli w kodzie programu jest ona inna. Nazwa węzła wpisana jest w konstruktorze tam gdzie pojawia się `super().__init__('goto_pose_action_server')`. Z tego powodu w zaprezentowanym przykładzie nazwą węzła będzie `goto_pose_action_server`.

Z prawej strony znajduje się odniesienie do lokalizacji pliku i miejsca w kodzie, z którego uruchamiany jest nowy węzeł. W tym przypadku jest to plik `go_to_pose_action_server.py` znajdujący się w katalogu `example_tsr_project`. Uruchomienie następuje w `main`.

3.6 Budowanie paczki

Należy przejść do głównego katalogu dla workspace'a (`example_tsr_ws`):

```

1 cd ~/tsr_workspaces/example_tsr_ws

```

następnie zbudować pojedynczą paczkę (można też zbudować cały workspace):

```

1 colcon build --symlink-install --packages-select example_tsr_project

```

i wykonać ponowny `source` w terminalu (lub uruchomić nowy terminal):

```

1 source install/setup.bash

```

3.7 Weryfikacja działania

W terminalu należy wpisać:

```

1 ros2 run example_tsr_project goto_pose_action_server

```

W wyniku (Rys. 3.2) działania polecenia utworzony został Serwer akcji o nazwie `goto_pose` obsługujący wiadomości typu `GoToPose`. Ponadto w terminalu wyświetla się informacja wysyłana do logów informująca o uruchomieniu serwera.

```

ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$ ros2 run example_tsr_project goto_pose_action_server
[INFO] [1741027608.672237760] [goto_pose_action_server]: Serwer akcji goto_pose uruchomiony.

```

Rys. 3.2: Wynik uruchomienia nowego węzła `goto_pose_action_server`

W nowym terminalu można sprawdzić listę aktywnych serwerów z akcjami i na liście powinien pojawić się `textttgoto_pose`:

```

1 ros2 action list

```

a następnie komendę umożliwiającą podgląd typu wiadomości obsługiwanej przez serwer akcji o nazwie `/goto_pose`.

```
1 ros2 action info -t /goto_pose
```

W wyniku wpisanych komend w terminalu powinien być wyświetlony następujący rezultat (Rys. 3.3):

```
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$ ros2 action list
/goto_pose
/turtle1/rotate_absolute
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$ ros2 action info -t /goto_pose
Action: /goto_pose
Action clients: 0
Action servers: 1
  /goto_pose_action_server [example_tsr_msgs/action/GoToPose]
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$
```

Rys. 3.3: Sprawdzenie poprawności działania węzła `goto_pose_action_server`

4 Procedura tworzenia nowego klienta dla akcji

Załóżmy, że mamy już istniejący pakiet (`example_tsr_project`) ROS2 kompilowany jako paczka Python i chcemy do niego dodać nowego klienta dla serwera akcji odpowiedzialnego za dojazd robota do celu. Jest to zmodyfikowana wersja węzła z ROS Services odpowiedzialnego za dojazd robota do punktów, które są dodawane przez serwis. Tym razem wykorzystana zostanie wiadomość dla akcji typu `GoToPose` z pakietu `example_tsr_msgs`.

4.1 Przejście do lokalizacji pakietu `example_tsr_project`

Przejdź do pakietu, w którym ma zostać utworzony nowy klient dla serwera akcji. W naszym przypadku będzie to pakiet z plikami projektowymi `example_tsr_project`.

```
1 cd ~/tsr_workspaces/example_tsr_ws/src/example_tsr_project
```

- `~` - oznacza lokalizację katalogu domowego
- `example_tsr_ws` - utworzony workspace. W katalogu `src` znajdują się wszystkie pakiety (nowe lub pobrane np. z githuba).

4.2 Utworzenie pliku `go_to_pose_action_client.py`

W pakiecie `example_tsr_project` należy utworzyć nowy plik ***go_to_pose_action_client.py*** w katalogu `example_tsr_project` (w kolejnym katalogu o tej samej nazwie a nie w głównym pakiecie gdzie znajdują się pliki `package.xml` i `setup.py`) a następnie nadać uprawnienia do wykonania. Brak uprawnień jest jednym z występujących problemów z widocznością węzła co uniemożliwia jego uruchomienie pomimo poprawnej konfiguracji paczki.

Należy wpisać kolejne polecenia w terminalu (należy znajdować się w głównej lokalizacji pakietu `example_tsr_project`)

```
1 touch example_tsr_project/go_to_pose_action_client.py
2 chmod +x example_tsr_project/go_to_pose_action_client.py
```

Nowo utworzony węzeł z klientem dla serwera akcji powinien mieć następującą lokalizację:

```
1 ~/tsr_workspaces/example_tsr_ws/src/example_tsr_project/example_tsr_project/go_to_pose_action_client.py
```

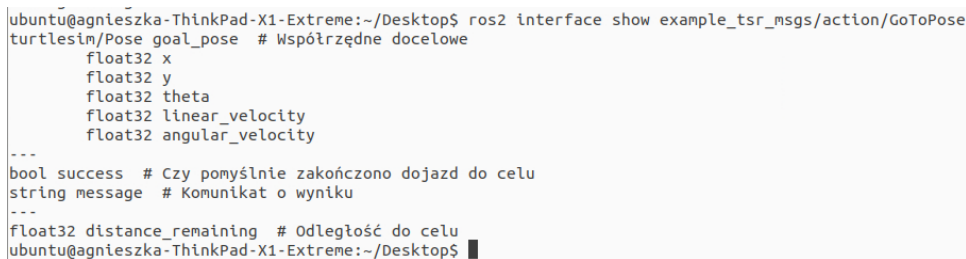
4.3 Sprawdzić typ wiadomości obsługiwanej przez serwer akcji

Następnie należy utworzyć nowy węzeł z klientem dla serwera akcji, który komunikuje się poprzez podanie nazwy (/goto_pose) serwera akcji. W tym przykładzie do komunikacji zastosowano wiadomość typu GoToPose z utworzonego pakietu example_tsr_msgs. Do utworzenia klienta niezbędna jest znajomość interfejsu komunikacyjnego (struktury przesyłanych wiadomości) obsługiwana przez serwer akcji. Typ wiadomości dla serwera akcji /goto_pose a następnie jej strukturę można odczytać z następujących poleceń:

```
1 ros2 action info -t /goto_pose
2 ros2 interface show example_tsr_msgs/action/GoToPose
```

Podobnie jak dla ROS Topic możemy w tym miejscu zastosować dowolny interfejs, ale później należy konsekwentnie wykonać zamianę nazw pakietu, z którego pochodzi wiadomość, zamianę typu i struktury obsługiwanej wiadomości na właściwą. W przypadku braku odpowiedniego interfejsu należy go utworzyć.

W wyniku operacji wyświetla się struktura wiadomości dla ROS Action (Rys. 3.1):



```
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$ ros2 interface show example_tsr_msgs/action/GoToPose
turtlesim/Pose goal_pose # Współrzędne docelowe
  float32 x
  float32 y
  float32 theta
  float32 linear_velocity
  float32 angular_velocity
---
bool success # Czy pomyślnie zakończono dojazd do celu
string message # Komunikat o wyniku
---
float32 distance_remaining # Odległość do celu
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$
```

Rys. 4.1: Struktura wiadomości dla akcji typu GoToPose

Znajomość typów i pól w wiadomości jest niezbędna do jej prawidłowego uzupełnienia.

4.4 Napisanie kodu dla węzła

Następnie należy napisać prosty węzeł z klientem wysyłającym cel do serwera akcji (**zalecane skopiowanie kodu z dołączonych plików lub ich wstawienie, gdyż zachowują wcięcia przy kopiowaniu**):

```
1 import rclpy
2 from rclpy.node import Node
3 from rclpy.action import ActionClient
4 from example_tsr_msgs.action import GoToPose # Import wiadomości dla akcji typu GoToPose z pakietu
   ↳ example_tsr_msgs
5 from turtlesim.msg import Pose
6
7
8 class GoToPoseActionClient(Node):
9     def __init__(self):
10         super().__init__('goto_pose_action_client')
11         # Utworzenie klienta dla serwera akcji, który obsługuje typ wiadomości GoToPose dla akcji o nazwie
   ↳ (na temacie) 'goto_pose'
12         self.action_client = ActionClient(self, GoToPose, 'goto_pose')
13
14     def send_goal(self, x, y):
15         """Wysyła cel do serwera akcji"""
16         # Utworzenie wiadomości z celem.
```

```

17     goal_msg = GoToPose.Goal()
18     goal_msg.goal_pose = Pose()
19     goal_msg.goal_pose.x = x
20     goal_msg.goal_pose.y = y
21
22     self.get_logger().info(f"Wysyłanie celu: ({x}, {y})")
23     # Blokuje kod do momentu, aż serwer akcji (Action Server) stanie się dostępny.
24     # Zastosowanie: Gdy klient akcji uruchamia się szybciej niż serwer i musi poczekać, aż serwer się
    ↪ podniesie.
25     self.action_client.wait_for_server()
26
27     # Wysyła żądanie wykonania akcji asynchronicznie (bez blokowania programu).
28     # Klient od razu przechodzi do następnej operacji, nie czekając na zakończenie akcji.
29     self.send_goal_future = self.action_client.send_goal_async(goal_msg,
    ↪ feedback_callback=self.feedback_callback)
30
31     # Po pewnym czasie serwer odpowiada, czy zaakceptował cel.
32     # Jeśli tak, przechodzimy do oczekiwania na wynik końcowy. Do metody self.goal_response_callback
33     self.send_goal_future.add_done_callback(self.goal_response_callback)
34
35     def goal_response_callback(self, future):
36         """Obsługuje odpowiedź serwera na żądanie celu"""
37         goal_handle = future.result()
38         if not goal_handle.accepted:
39             self.get_logger().info("Cel odrzucony przez serwer.")
40             return
41
42         self.get_logger().info("Cel zaakceptowany, czekam na wynik...")
43
44         # Uruchamia asynchroniczne oczekiwanie na wynik akcji (goal_handle.get_result_async()).
45         # Dzięki temu program nie blokuje się i może w międzyczasie obsługiwać inne operacje (np. otrzymywać
    ↪ feedback z serwera).
46         self.get_result_future = goal_handle.get_result_async()
47
48         # Gdy serwer akcji zakończy wykonywanie zadania, ta linia wywoła result_callback().
49         # result_callback() odbierze wynik akcji i wyświetli komunikat końcowy. Przejdźcie do metody
    ↪ self.result_callback
50         self.get_result_future.add_done_callback(self.result_callback)
51
52     def feedback_callback(self, feedback_msg):
53         """Obsługuje odbiór feedbacku o postępie"""
54         self.get_logger().info(f"Pozostała odległość do celu:
    ↪ {feedback_msg.feedback.distance_remaining:.2f}m")
55
56     def result_callback(self, future):
57         """Obsługuje odbiór wyniku końcowego akcji"""
58         result = future.result().result
59         self.get_logger().info(f"Wynik akcji: {result.message} (Sukces: {result.success})")
60
61
62     def main():
63         # Inicjalizacja ROS2 dla programu, musi być wykonana przed utworzeniem jakiegokolwiek węzła (node'a).
64         rclpy.init()
65         # Utworzenie nowego obiektu węzła ROS2 na podstawie klasy GoToPoseActionClient
66         node = GoToPoseActionClient()
67
68         # Wysyłanie celu (np. x=5.0, y=3.0)
69         node.send_goal(5.0, 3.0)
70
71         # Spin umożliwia węzłowi ROS2 działanie i oczekuje na zdarzenia. Brak linii spowoduje natychmiastowe
    ↪ zakończenie działania programu.
72         rclpy.spin(node)
73         # Usuwa węzeł przed zamknięciem programu. Zwalnia zasoby pamięci zajmowane przez ten obiekt.
74         node.destroy_node()
75         # Zamyka połączenie z ROS2.
76         rclpy.shutdown()
77
78     if __name__ == '__main__':
79         main()
80

```

4.5 Aktualizacja *setup.py*

W pakiecie *example_tsr_project* w pliku *setup.py*. Należy zmodyfikować linię z *entry_points* tak, aby dodać nowy węzeł odpowiedzialny za uruchomienie serwera akcji, który został przed chwilą utworzony. W tym miejscu należy dodawać uruchomienie każdego nowego węzła (node).

```

1     entry_points={
2         'console_scripts': [
3             'goto_pose_action_client = example_tsr_project.go_to_pose_action_client:main',
4             ],
5     }

```

Lewa strona w linii z dodanym `goto_pose_action_client` odnosi się do unikalnej nazwy w pakiecie, która

będzie wykorzystywana podczas uruchamiania nowego węzła. Ta nazwa może nie być widoczna po uruchomieniu węzła na liście uruchomionych węzłów (`ros2 node list`), jeśli w kodzie programu jest ona inna. Nazwa węzła wpisana jest w konstruktorze tam gdzie pojawia się

`super().__init__('goto_pose_action_client')`. Z tego powodu w zaprezentowanym przykładzie nazwą węzła będzie `goto_pose_action_client`.

Z prawej strony znajduje się odniesienie do lokalizacji pliku i miejsca w kodzie, z którego uruchamiany jest nowy węzeł. W tym przypadku jest to plik `go_to_pose_action_client.py` znajdujący się w katalogu `example_tsr_project`. Uruchomienie następuje w `main`.

4.6 Budowanie paczki

Należy przejść do głównego katalogu dla workspace'a (`example_tsr_ws`):

```
1 cd ~/tsr_workspaces/example_tsr_ws
```

następnie zbudować pojedynczą paczkę (można też zbudować cały workspace):

```
1 colcon build --symlink-install --packages-select example_tsr_project
```

i wykonać ponowny `source` w terminalu (lub uruchomić nowy terminal):

```
1 source install/setup.bash
```

4.7 Weryfikacja działania

Przyjęto założenie, że symulacja `turtlesim` jest uruchomiona. Do poprawnego działania wymagane jest dalsze działanie serwera od obsługi akcji `/goto_pose`. Jeśli nie jest uruchomiony to należy to zrobić:

```
1 ros2 run example_tsr_project goto_pose_action_server
```

Aby uruchomić klienta w terminalu należy wpisać:

```
1 ros2 run example_tsr_project goto_pose_action_client
```

W wyniku (Rys. 4.2) działania polecenia uruchomiony został klient dla serwera akcji o nazwie `goto_pose` wysyłający wiadomość z celem typu `GoToPose`. Ponadto w terminalu wyświetla się informacja wysyłana do logów informująca o wysłaniu robota do celu.

```
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$ ros2 run example_tsr_project goto_pose_action_client
[INFO] [1741029413.581794349] [goto_pose_action_client]: Wysyłanie celu: (5.0, 3.0)
[INFO] [1741029413.840697594] [goto_pose_action_client]: Cel zaakceptowany, czekam na wynik...
[INFO] [1741029413.859024145] [goto_pose_action_client]: Wynik akcji: Robot dotarł do celu! (Sukces: True)
```

Rys. 4.2: Wynik uruchomienia nowego węzła `goto_pose_action_client`

5 Dokumentacja i inne linki

Rozdział może zawierać dokumentację poleceń, linki do źródeł na podstawie, których powstała instrukcja oraz inne linki pozwalające poszerzyć wiedzę.

— Dokumentacja ROS Humble

<https://docs.ros.org/en/humble/index.html>

— Więcej o akcjach

<https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Actions/Understanding-ROS2-Actions.html>

```
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$ ros2 action --help
usage: ros2 action [-h]
                  Call `ros2 action <command> -h` for more detailed usage.
...

Various action related sub-commands

options:
  -h, --help            show this help message and exit

Commands:
  info                  Print information about an action
  list                  Output a list of action names
  send_goal             Send an action goal

  Call `ros2 action <command> -h` for more detailed usage.
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$
```

Rys. 5.1: Dokumentacja polecenia `ros2 action`